Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W2-01 Wie oft hat Aria gespielt? · Fehler im Java-Code

Fehler 1: Start bei i = 1 – der Eintrag mit Index 0 (Platz 1!) wird nie geprüft. Korrektur: i = 0.

Fehler 2: Bedingung i <= anzahl – im letzten Durchlauf wird eintraege[anzahl] gelesen. Dieses Feld ist null → NullPointerException; bei voller Liste liegt der Index sogar außerhalb der Reihung. Korrektur: i < anzahl.

Fehler 3: == vergleicht bei Zeichenketten die Referenzen, nicht den Inhalt. Inhaltsgleiche Namen aus verschiedenen Zeichenketten-Objekten (z. B. aus einer Eingabe) werden nicht gezählt. Korrektur: equals. (Die Online-IDE vergleicht mit == inhaltlich – im Abitur und in Java gilt: equals!)

```
public int anzahlSpiele(String name) {
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getName().equals(name)) {
            zaehler++;
        }
    }
    return zaehler;
}
```

W2-02 Die Minusrunde · Fehler im Java-Code

Fehler 1: max = 0 – haben alle Einträge negative Punkte, z. B. Aria −30, Bruno −10, ist keine Bedingung wahr, es wird null statt Bruno geliefert. Korrektur: mit dem ersten Eintrag beginnen (bester = eintraege[0]) – das erfordert vorher die Leer-Abfrage.

Fehler 2: i <= anzahl – eintraege[anzahl] ist null (NullPointerException), bei voller Liste Index außerhalb der Reihung. Wirkt bei jeder Liste. Korrektur: i < anzahl.

Fehler 3: >= statt > – bei Gleichstand wird der letzte statt der erste geliefert, z. B. Aria 500, Bruno 500 → Bruno. Korrektur: >.

Leer-Abfrage: Wer mit dem ersten Eintrag startet (z. B. max = eintraege[0].getPunkte()), löst bei leerer Liste eine NullPointerException aus, weil eintraege[0] null ist. Deshalb zuerst: if (anzahl == 0) return null;

```
public Eintrag besterEintrag() {
    if (anzahl == 0) {
        return null;
    }
    Eintrag bester = eintraege[0];
    for (int i = 1; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > bester.getPunkte()) {
            bester = eintraege[i];
        }
    }
    return bester;
}
```

W2-03 Die Späher der Rangliste · Methode implementieren

Kernpunkte: a) Zähler mit 0 initialisieren · Schleife von 0 bis anzahl − 1 (nicht eintraege.length – sonst null-Zugriff) · echter Vergleich > (nicht >=) · Rückgabe nach der Schleife.

b) Vergleich mit equals · return i sofort beim ersten Treffer (erstes Vorkommen) · return -1 erst NACH der Schleife (nicht im else-Zweig!).

```
public int anzahlUeber(int grenze) {
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > grenze) {
            zaehler++;
        }
    }
    return zaehler;
}

public int indexVon(String name) {
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getName().equals(name)) {
            return i;
        }
    }
    return -1;
}
```

W2-04 Das Doppelgänger-Rätsel · Struktogramm lesen

a) Vergleiche: (0,1) Aria/Bruno · (0,2) Aria/Cem · (0,3) Aria/Bruno · (1,2) Bruno/Cem · (1,3) Bruno/Bruno → gleich. Rückgabewert: wahr (nach 5 Vergleichen).

b) Die Operation prüft, ob ein Name mindestens zweimal in der Liste vorkommt (hatDoppelteNamen). Jedes Paar von Einträgen wird genau einmal verglichen, weil j immer rechts von i beginnt.

c) 45 Vergleiche: Für i = 0 gibt es 9 Partner, für i = 1 noch 8, …, für i = 8 noch 1. 9 + 8 + … + 1 = 10 · 9 / 2 = 45. Allgemein n(n − 1)/2 – bei doppelt so vielen Einträgen also etwa viermal so viele Vergleiche.

W2-05 Selectionsort: Die Auswahl der Besten · Tracetabelle

b) 4 + 3 + 2 + 1 = 10 Vergleiche, 2 Vertauschungen (bei i = 0 und i = 2).

c) Ebenfalls 10. Die innere Schleife läuft immer vollständig bis zum Ende, um das Maximum sicher zu finden – unabhängig von der Belegung. Selectionsort braucht stets n(n − 1)/2 Vergleiche; nur die Zahl der Vertauschungen sinkt (hier auf 0).

```
i | maxIndex | Reihung nach dem Durchlauf
--+----------+----------------------
- |    -     | [40, 70, 20, 90, 50]
0 |    3     | [90, 70, 20, 40, 50]
1 |    1     | [90, 70, 20, 40, 50]
2 |    4     | [90, 70, 50, 40, 20]
3 |    3     | [90, 70, 50, 40, 20]
```

W2-06 Insertionsort: Einsortieren wie beim Kartenspiel · Tracetabelle

Hinweis: Bei j = 0 bricht && sofort ab – dann wird r[j − 1] < r[j] nicht mehr ausgewertet und nicht gezählt (Durchlauf i = 3).

b) 1 + 2 + 3 + 3 = 9 Vergleiche (7 Vertauschungen). Insertionsort spart hier einen Vergleich, weil die innere Schleife abbricht, sobald das Element an seinem Platz ist.

c) Jede bereits absteigend sortierte Belegung, z. B. [90, 80, 60, 50, 30]: pro Durchlauf genau 1 Vergleich (falsch), also n − 1 = 4 Vergleiche und 0 Vertauschungen.

```
i | Vergleiche | Vertauschungen | Reihung nach dem Durchlauf
--+------------+----------------+----------------------
- |     -      |       -        | [30, 80, 50, 90, 60]
1 |     1      |       1        | [80, 30, 50, 90, 60]
2 |     2      |       1        | [80, 50, 30, 90, 60]
3 |     3      |       3        | [90, 80, 50, 30, 60]
4 |     3      |       2        | [90, 80, 60, 50, 30]
```

W2-07 Die Innenschleife · Lückencode

a) (1) i · (2) j > 0 · (3) j - 1 · (4) j-- bzw. j = j - 1

b) && wertet von links aus und bricht ab, sobald ein Teil falsch ist (Kurzschlussauswertung). Ist j = 0, wird besser(…) nicht mehr aufgerufen – sonst würde auf eintraege[-1] zugegriffen (Index außerhalb der Reihung).

c) besser vergleicht mit > (echt größer). Bei gleicher Punktzahl liefert besser falsch, die innere Schleife bricht ab – der hintere Eintrag wird nie an einem gleich guten vorbeigeschoben.

```
public void insertionSort() {
    for (int i = 1; i < anzahl; i++) {
        int j = i;
        while (j > 0 && besser(eintraege[j], eintraege[j - 1])) {
            vertausche(j, j - 1);
            j--;
        }
    }
}
```

W2-08 Bubblesort mit Notbremse · Tracetabelle

b) 4 + 3 + 2 = 9 Vergleiche. Im 3. Durchlauf wird nicht mehr getauscht → getauscht bleibt falsch → Abbruch.

c) Bester Fall: schon absteigend sortiert, z. B. [90, 70, 50, 40, 20] → ein Durchlauf ohne Tausch: n − 1 = 4 Vergleiche. Schlechtester Fall: aufsteigend sortiert, z. B. [20, 40, 50, 70, 90] → 4 + 3 + 2 + 1 = 10 = n(n − 1)/2 Vergleiche (und 10 Vertauschungen).

```
Durchlauf | Vertauschungen | Reihung nach dem Durchlauf
----------+----------------+----------------------
    -     |       -        | [90, 50, 70, 80, 20]
    1     |       2        | [90, 70, 80, 50, 20]
    2     |       1        | [90, 80, 70, 50, 20]
    3     |       0        | [90, 80, 70, 50, 20]  → Abbruch
```

W2-09 Neuer Highscore! · Methode implementieren

Kernpunkte: Abfrage e == null oder voll ganz am Anfang · Schleife von hinten (i = anzahl) mit i > 0 als ERSTER Teilbedingung · verschieben mit eintraege[i] = eintraege[i - 1] · besser(e, …) mit echtem > sorgt dafür, dass e hinter gleich gute Einträge kommt · danach einsetzen, anzahl erhöhen, true zurückgeben.

Typische Fehler: Schleife von vorne → Einträge werden überschrieben; Einfügen ganz vorne/hinten vergessen zu testen (leere Liste!); anzahl++ vergessen.

```
public boolean einfuegenSortiert(Eintrag e) {
    if (e == null || anzahl == eintraege.length) {
        return false;
    }
    int i = anzahl;
    while (i > 0 && besser(e, eintraege[i - 1])) {
        eintraege[i] = eintraege[i - 1];
        i--;
    }
    eintraege[i] = e;
    anzahl++;
    return true;
}
```

W2-10 Andersherum und mitgezählt · Verändern und Erweitern

a) Nur der Vergleich ändert sich: r[j] > r[j + 1] (größeres Element wandert nach hinten).

b) Zähler vor den Schleifen mit 0 initialisieren, bei jedem Tausch erhöhen, nach den Schleifen zurückgeben; Rückgabetyp int statt void.

c) 6. Jedes der 3 + 2 + 1 = 6 Vergleichspaare führt zum Tausch. Mehr geht nicht: Es gibt bei 4 Elementen nur 4 · 3 / 2 = 6 Vergleiche, und jeder Vergleich kann höchstens einen Tausch auslösen.

```
public static int sortiere(int[] r) {
    int tausch = 0;
    for (int d = 0; d < r.length - 1; d++) {
        for (int j = 0; j < r.length - 1 - d; j++) {
            if (r[j] > r[j + 1]) {
                int hilf = r[j];
                r[j] = r[j + 1];
                r[j + 1] = hilf;
                tausch++;
            }
        }
    }
    return tausch;
}
```

W2-11 Halbieren statt Durchsuchen · Code ↔ Struktogramm

Prüfe dein Struktogramm: Signatur binaer(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl · Zuweisungen mit ← · Bedingung lo ≤ hi · ganzzahlige Division (als Kommentar vermerken) · Vergleich mit = (nicht ==) · „gib −1 zurück“ als letzter Block NACH der Schleife.

Voraussetzung: r ist aufsteigend sortiert – sonst kann die Hälfte, in der x liegt, verworfen werden.

```
binaer(r: Reihung von Ganzzahl, x: Ganzzahl): Ganzzahl
  lo ← 0
  hi ← länge(r) − 1
  solange lo ≤ hi
    mid ← (lo + hi) / 2     // ganzzahlige Division
    falls r[mid] = x
      dann: gib mid zurück
      sonst: falls r[mid] < x
               dann: lo ← mid + 1
               sonst: hi ← mid − 1
  gib −1 zurück
```

W2-12 Die wackelige Blase · Fehler im Struktogramm

Fehler 1: Innere Grenze länge(r) − 1 − d – für d = 0 ist j = länge(r) − 1, dann liegt r[j + 1] außerhalb der Reihung. Korrektur: bis länge(r) − 2 − d.

Fehler 2: r[j] > r[j + 1] schiebt das größere Element nach hinten – das sortiert aufsteigend. Korrektur: r[j] < r[j + 1].

Fehler 3: Tausch ohne Hilfsvariable – nach r[j] ← r[j + 1] ist der alte Wert von r[j] verloren, beide Felder enthalten danach denselben Wert. Korrektur: hilf ← r[j]; r[j] ← r[j + 1]; r[j + 1] ← hilf.

```
blasen(r: Reihung von Ganzzahl): nichts
  für d von 0 bis länge(r) − 2
    für j von 0 bis länge(r) − 2 − d
      falls r[j] < r[j + 1]
        dann: hilf ← r[j]
              r[j] ← r[j + 1]
              r[j + 1] ← hilf
        sonst: –
```

W2-13 Wie lange dauert das Ranking? · Funktionsweise erläutern

a) Sortieren durch Auswahl (Selectionsort), hier mit dem Minimum: Im unsortierten Bereich r[0] bis r[ende] wird der Index m des kleinsten Elements gesucht und dieses mit r[ende] vertauscht. Danach ist r[ende] endgültig richtig, der unsortierte Bereich wird um eins kleiner. Weil die kleinsten Werte nach hinten wandern, entsteht eine absteigende Sortierung.

b) 4 + 3 + 2 + 1 = 10 Vergleiche; allgemein (n − 1) + … + 1 = n(n − 1)/2. Unabhängig von der Belegung – die innere Schleife läuft immer vollständig.

c) Die Vergleichszahl wächst etwa mit n^2: doppelt so viele Einträge → etwa viermal so viele Vergleiche. 20 000 → ca. 4 · 0,3 s = 1,2 s; 40 000 → ca. 16 · 0,3 s = 4,8 s.

W2-14 Die Halbierungs-Jagd · Tracetabelle

a) binaer(r, 47) liefert 9 (3 Durchläufe). binaer(r, 50) liefert −1 (4 Durchläufe; danach lo = 10 > hi = 9).

b) 4 Durchläufe (11 → 5 → 2 → 1 Elemente bleiben übrig).

c) Jeder Durchlauf halbiert den Suchbereich mindestens: 100 → höchstens 50 → 25 → 12 → 6 → 3 → 1. Spätestens im 7. Durchlauf ist nur noch ein Element übrig, danach ist der Bereich leer. Allgemein: höchstens $\lfloor \log_2 n \rfloor + 1$ Durchläufe ($2^6 = 64 \le 100 < 128 = 2^7$). Bei n = 200 höchstens 8 – eine Verdopplung kostet nur einen Durchlauf mehr.

```
binaer(r, 47)                    binaer(r, 50)
lo | hi | mid | r[mid]           lo | hi | mid | r[mid]
---+----+-----+-------           ---+----+-----+-------
 0 | 10 |  5  |  25               0 | 10 |  5  |  25
 6 | 10 |  8  |  41               6 | 10 |  8  |  41
 9 | 10 |  9  |  47 → gib 9       9 | 10 |  9  |  47
                                 10 | 10 | 10  |  52
Rückgabe: 9                      Rückgabe: −1
```